

Programowanie równoległe	Kierunek: Informatyka techniczna	Grupa: 9
Imię i nazwisko: Jakub Świerczyński	Tematy: Lab11 – MPI - podstawy Lab12 – MPI – komunikacja grupowa Lab13 – Pomiary wydajności	Termin oddania: 18.01.2025

Laboratorium 11

· Program MPI_simple

Procesy o randze większej niż 0 wysyłają do procesu 0 informację o swojej randze oraz nazwę hosta (pobraną funkcją `gethostname`).

```
jakub@DESKTOP-IVF3VM7:/mnt/c/Rownolegle/lab11/simple$ make run
mpiexec -oversubscribe -np 4 ./MPI_simple
Odebrano rangę: 3 od procesu o randze 3 działającego na hoście: DESKTOP-IVF3VM7
Odebrano rangę: 2 od procesu o randze 2 działającego na hoście: DESKTOP-IVF3VM7
Odebrano rangę: 1 od procesu o randze 1 działającego na hoście: DESKTOP-IVF3VM7
```

· Program Sztafeta

Każdy proces ustala $prev = (rank - 1 + size) \% size$ oraz $next = (rank + 1) \% size$, tworząc pierścień.

Proces 0 inicjuje komunikat (liczba 100), wysyła do procesu 1, który modyfikuje dane i przekazuje dalej.

W wariancie zamkniętym ostatni proces odsyła dane do procesu 0.

```
jakub@DESKTOP-IVF3VM7:/mnt/c/Rownolegle/lab11/sztafeta$ make run
mpiexec -oversubscribe -np 4 ./sztafeta
Proces 0 wysyła liczbę 100 do procesu 1
Proces 1 odebrał liczbę 100 od procesu 0
Proces 1 wysyła liczbę 101 do procesu 2
Proces 2 odebrał liczbę 101 od procesu 1
Proces 2 wysyła liczbę 102 do procesu 3
Proces 3 odebrał liczbę 102 od procesu 2
Proces 0 odebrał liczbę 103 od procesu 3 (zamknięty pierścien)
```

· Program Struktura

Zdefiniowano strukturę zawierającą 3 typy danych (int, double oraz tablice znaków).

```
typedef struct {
    int number;
    double value;
    char name[20];
} MyData;
```

Dane są pakowane funkcją `MPI_Pack` i wysyłane w pierścieniu (podobnie jak w „sztafecie”).

```
MPI_Pack(&data.number, 1, MPI_INT, buffer, buf_size, &position, MPI_COMM_WORLD);
MPI_Pack(&data.value, 1, MPI_DOUBLE, buffer, buf_size, &position, MPI_COMM_WORLD);
MPI_Pack(data.name, strlen(data.name)+1, MPI_CHAR, buffer, buf_size, &position, MPI_COMM_WORLD);
```

Każdy proces wyświetla odebrane dane, modyfikuje je (zwiększa numer, value oraz dodaje „X” do imienia) i wysyła do następnego.

```
MPI_Unpack(recv_buf, 100, &recv_pos, &receivedData.number, 1, MPI_INT, MPI_COMM_WORLD);
MPI_Unpack(recv_buf, 100, &recv_pos, &receivedData.value, 1, MPI_DOUBLE, MPI_COMM_WORLD);
MPI_Unpack(recv_buf, 100, &recv_pos, receivedData.name, 20, MPI_CHAR, MPI_COMM_WORLD);

printf("Proces %d otrzymał dane od procesu %d, dane: number=%d, value=%.2f, name=%s\n", rank, prev,
receivedData.number, receivedData.value, receivedData.name);

receivedData.number += 1;
receivedData.value += 0.5;
int len = strlen(receivedData.name);
if (len < 19) {
    receivedData.name[len] = 'X';
    receivedData.name[len+1] = '\0';
}

printf("Proces %d zmodyfikował dane: number=%d, value=%.2f, name=%s\n", rank, receivedData.number,
receivedData.value, receivedData.name);

int new_buf_size = sizeof(int) + sizeof(double) + strlen(receivedData.name) + 1;
char *send_buffer = malloc(new_buf_size);
int send_pos = 0;
MPI_Pack(&receivedData.number, 1, MPI_INT, send_buffer, new_buf_size, &send_pos, MPI_COMM_WORLD);
MPI_Pack(&receivedData.value, 1, MPI_DOUBLE, send_buffer, new_buf_size, &send_pos, MPI_COMM_WORLD);
MPI_Pack(receivedData.name, strlen(receivedData.name)+1, MPI_CHAR, send_buffer, new_buf_size, &send_pos,
MPI_COMM_WORLD);
```

```
jakub@DESKTOP-IVF3VM7:/mnt/c/Rownolegle/lab11/struktura$ make run
mpiexec -oversubscribe -np 4 ./struktura

Proces 0 (inicjalizacja): number=42, value=3.14, name=Jakub
Proces 0 wysyła dane do procesu 1

Proces 1 otrzymał dane od procesu 0, dane: number=42, value=3.14, name=Jakub
Proces 1 zmodyfikował dane: number=43, value=3.64, name=JakubX
Proces 1 wysyła dane do procesu 2

Proces 2 otrzymał dane od procesu 1, dane: number=43, value=3.64, name=JakubX
Proces 2 zmodyfikował dane: number=44, value=4.14, name=JakubXX
Proces 2 wysyła dane do procesu 3

Proces 3 otrzymał dane od procesu 2, dane: number=44, value=4.14, name=JakubXX
Proces 3 zmodyfikował dane: number=45, value=4.64, name=JakubXXX
Proces 0 otrzymał dane od procesu 3, dane: number=45, value=4.64, name=JakubXXX
```

Wnioski

Za pomocą funkcji `gethostname()` i `MPI_Send/MPI_Recv` można przekazywać zarówno dane proste, jak i tablice znaków.

Wykorzystanie `MPI_PACKED` umożliwia wysyłanie złożonych struktur w sposób elastyczny.

Konwencja SPMD upraszcza pisanie programu: wszystkie procesy wykonują ten sam kod, a zachowanie różnicuje się przez `rank`.

Labratorium 12

· Program obliczania liczby π (`MPI_pi.c`)

Wczytanie liczby wyrazów (`N`) przez proces o `randze 0`.

Rozgłoszenie wartości `N` (`MPI_Bcast`) do pozostałych procesów.

```
if (rank==0){
    printf("Podaj maksymalna liczbe wyrazow do obliczenia przyblizenia PI\n");
    scanf("%d", &N);
}
MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Każdy proces ustala swój zakres iteracji (blokowo) i oblicza lokalne sumy (`local_suma_plus` i `local_suma_minus`).

```
int block_size = N / size;
int my_start = rank * block_size;
int my_end = (rank + 1) * block_size;

if (rank == size - 1) {
    my_end = N;
}

SCALAR local_suma_plus = 0.0;
SCALAR local_suma_minus = 0.0;

for (int i = my_start; i < my_end; i++) {
    int j = 1 + 4*i;
    local_suma_plus += 1.0 / j;
    local_suma_minus += 1.0 / (j + 2.0);
}
```

Sprowadzenie (redukcja) sum lokalnych do procesu 0 (`MPI_Reduce`).

```
SCALAR global_suma_plus = 0.0;
SCALAR global_suma_minus = 0.0;
MPI_Reduce(&local_suma_plus, &global_suma_plus, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&local_suma_minus, &global_suma_minus, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Proces 0 wylicza przybliżenie π i wyświetla wyniki (wraz z różnicą względem M_PI).

```
if(rank == 0) {
    SCALAR pi_approx = 4.0 * (global_suma_plus - global_suma_minus);
    double diff = M_PI - pi_approx;
    printf("PI obliczone:  %20.15lf\n", pi_approx);
    printf("PI prawdziwe:  %20.15lf\n", M_PI);
    printf("Roznica:      %20.15lf\n", diff);
}
```

```
jakub@DESKTOP-IVF3VM7:/mnt/c/Rownolegle/lab12/MPI_pi$ make run
mpiexec -oversubscribe -np 4 ./MPI_pi
Podaj maksymalna liczbe wyrazow do obliczenia przyblizenia PI
2000
PI obliczone:      3.141342653593684
PI prawdziwe:      3.141592653589793
Roznica:           0.000249999996109
```

·Mnożenie macierz–wektor (mat_vec_row_MPI.c)

Proces 0 inicjuje dane: macierz A (wielkości WYMIAR × WYMIAR), wektor x i wektor y (dla obliczeń sekwencyjnych).

Wersja sekwencyjna wykonuje się tylko dla procesu 0 do weryfikacji poprawności.

W równoległej dekompozycji wierszowej MPI_Scatter rozsyła fragmenty macierzy i wektora do poszczególnych procesów.

```
MPI_Scatter(a, n_wier*WYMIAR, MPI_DOUBLE, a_local, n_wier*WYMIAR, MPI_DOUBLE, root, MPI_COMM_WORLD);
MPI_Scatter(x, n_wier, MPI_DOUBLE, &x[rank*n_wier], n_wier, MPI_DOUBLE, root, MPI_COMM_WORLD);
```

Każdy proces oblicza lokalny wynik.

Wyniki są zbierane (za pomocą MPI_Gather).

```
MPI_Gather(z, n_wier, MPI_DOUBLE, z, n_wier, MPI_DOUBLE, root, MPI_COMM_WORLD);
```

```
jakub@DESKTOP-IVF3VM7:/mnt/c/Rownolegle/lab12/mat_vec_row_MPI$ make
mpicc -c -O2 -fopenmp mat_vec_row_MPI.c
mpicc -O2 -fopenmp mat_vec_row_MPI.o -o mat_vec_row_MPI -lm
mpiexec -np 4 --oversubscribe ./mat_vec_row_MPI
poczatek (wykonanie sekwencyjne)
    czas wykonania (zaburzony przez MPI?): 0.080386, Gflop/s: 2.527953, GB/s> 10.112814
Starting MPI matrix-vector product with block row decomposition!
Wersja rownolegla MPI z dekompozycja wierszowa blokowa
    czas wykonania: 0.039448, Gflop/s: 5.151398, GB/s> 20.607636
Starting MPI matrix-vector product with block column decomposition!
Werja rownolegla MPI z dekompozycja kolumnowa blokowa
    czas wykonania: 0.051641, Gflop/s: 3.935133, GB/s> 15.742094
Blad! i=0, y[i]=170698750320.000000, z[i]=0.000000 - complete the code for column decomposition
```

Wnioski

Komunikacja grupowa w MPI (np. MPI_Scatter, MPI_Gather, MPI_Bcast, MPI_Reduce) zakłada, że wszystkie procesy wywołują tę samą operację, a ranga procesu określa fragment danych, jaki jest odbierany/wysyłany.

W komunikacji punkt-punkt (MPI_Send, MPI_Recv) mamy większą swobodę co do rozsyłania danych: można np. wysłać dowolny zestaw elementów do dowolnego procesu.

Komunikacja grupowa upraszcza kod (mniej wywołań send/recv) i poprawia czytelność.

Laboratorium 13

W katalogu calka skompilowaliśmy kod calka_omp.c (metoda trapezów) z flagą -fopenmp i uruchamialiśmy go z różną liczbą wątków (1, 2, 4, 6) sterowaną zmienną środowiskową OMP_NUM_THREADS.

W katalogu mat_vec rozpakowaliśmy paczkę mat_vec_row_MPI_perf.tgz i użyliśmy make run, zmieniając liczbę procesów (1, 2, 4, 6) za pomocą opcji -np w mpiexec.

Obliczanie przyspieszenia:

$$S(p) = \frac{T_p(1)}{T_p(p)}$$

p – liczba procesów.

$S(p)$ – przyspieszenie dla p procesów.

$T(1)$ – czas dla 1 procesu.

$T(p)$ – czas dla p procesów.

Obliczanie efektywności zrównoleglenia:

$$E(p) = \frac{S(p)}{p} = \frac{T(1)}{p \cdot T(p)}$$

$E(p)$ – efektywność dla p procesów.

Wyniki i analiza

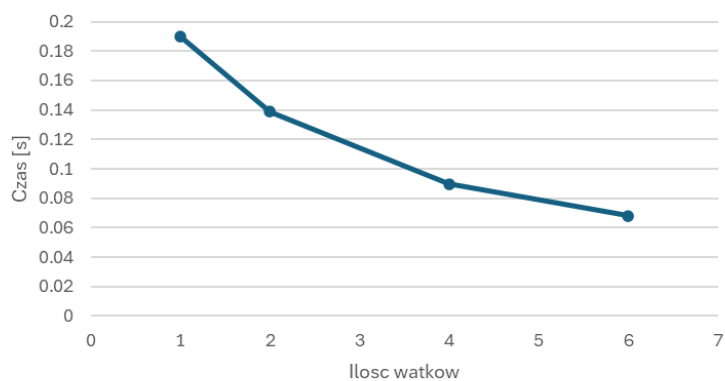
Program calka_omp (metoda trapezów)

Liczba wątków	Średni czas [s]	Przyspieszenie	Efektywność
1	0.190056	1.0	1.0
2	0.138775333	1.369522922	0.684761461
4	0.089819333	2.115980969	0.528995242
6	0.068139667	2.789212353	0.464868726

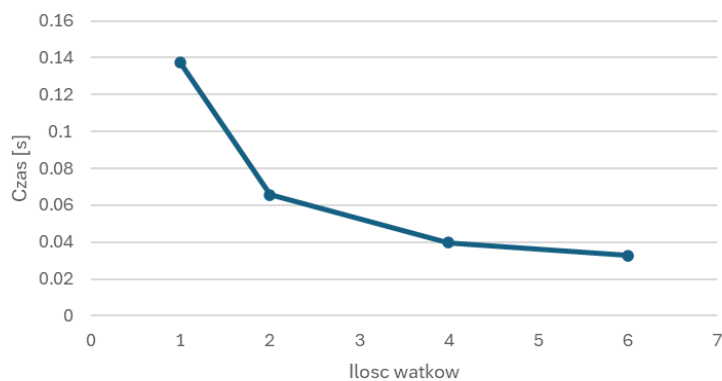
Program mat_vec_row_MPI (mnożenie macierz–wektor z użyciem MPI)

Liczba wątków	Średni czas [s]	Przyspieszenie	Efektywność
1	0.1376	1.0	1.0
2	0.065696333	2.094485233	1.047242616
4	0.039701333	3.46587856	0.86646964
6	0.032879	4.185042124	0.697507021

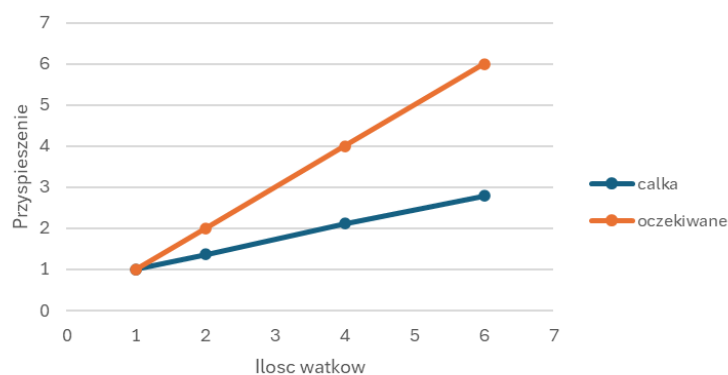
Wykres czasu dla liczenia całki



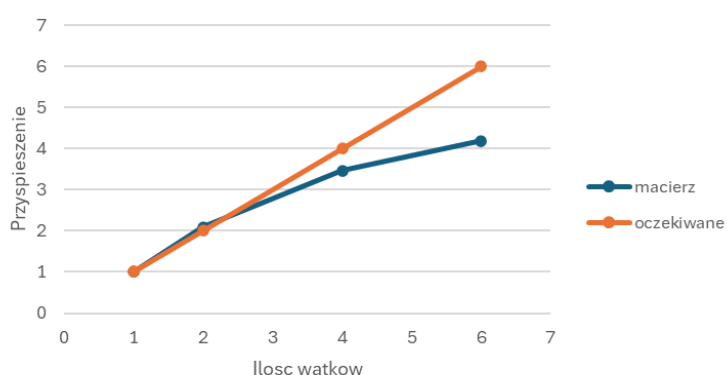
Wykres czasu dla macierzy



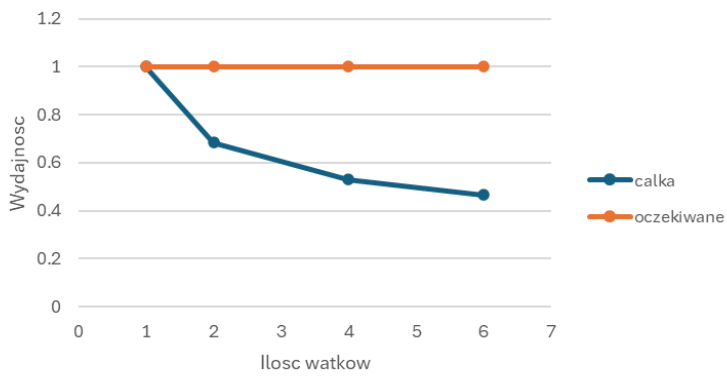
Przyspieszenie dla całki



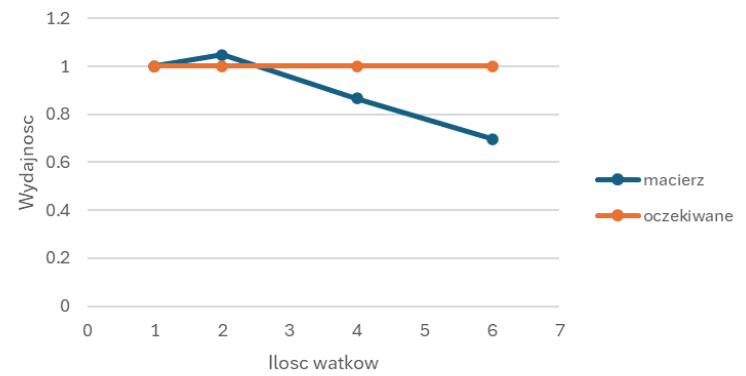
Przyspieszenie dla macierzy



Wydajność dla całki



Wydajność dla macierzy



Wnioski

Przeprowadzone pomiary (każdy co najmniej trzykrotny) pozwoliły obliczyć średni czas, przyspieszenie i efektywność.

Zarówno dla `całka_omp`, jak i `mat_vec_row_MPI` obserwujemy poprawę wydajności wraz ze wzrostem liczby wątków/procesów, choć daleką od ideału ($\text{speedup} < \text{liczba wątków}$).

Powodem odchylenia są typowe narzuty (synchronizacja, komunikacja, fragmenty sekwencyjne).

Wyniki pokazały, że już przy 4–6 jednostkach równoległych zyskujemy kilkukrotnie szybsze wykonanie w porównaniu do wersji 1-wątkowej/1-procesowej.

